Interaktív és vizualizációs oktatási felület a Számításelmélet alapjai I–II. tárgyhoz

Szabó Anna

2026

Tartalomjegyzék

1. fejezet

Bevezetés

1.1. Motiváció és problémafelvetés

Az egyetemi évek alatt számos olyan tárgyat tanulunk, amelyek megértése jelentős absztrakciós képességet igényel, és sokszor nagyon elvontnak, nehezen megfoghatónak tűnnek. Nekem is voltak ilyen tárgyak s ez igaz volt Számításelmélet alapjai I–II. tárgyra, amely olyan témaköröket dolgoz fel, mint a formális nyelvek, a grammatikák, a véges automaták, a reguláris kifejezések, a veremautomaták vagy a Turing-gépek. Ezek a fogalmak ószámunkra a hallgatók számára gyakran nehezen értelmezhetők, mivel működésük és viselkedésük hagyományos tananyagok segítségével nem mindig szemléltethető egyszerűen.

A tananyag elsajátításához általában előadásanyagok, prezentációk és órai jegyzetek állnak rendelkezésünkre. Bár ezek fontos szerepet töltenek be az oktatásban és ezek adják a tudásunk alapját, a mélyebb megértés sokszor a gyakorlatokra, illetve az önálló otthoni tanulásra hárul.

A problémát ott azonosítottam, ahol egy absztrakt elméleti folyamat megértéséhez több, egymásra épülő állapotot vagy lépést kell követni. Sokszor papíron végig követni ezeket ahhoz vezetett, hogy az ember megpróbált minden állapotot külön papírra rajzolni. Ami sokszor időigényes volt, de ez adta a mély megértés kulcsát. Ezért az ilyen témáknál mi, hallgatók gyakran nagyon sok időt töltünk az anyag önálló feldolgozásával.

A szakdolgozat erre a problémára kíván megoldást nyújtani egy olyan webalapú oktatási rendszer fejlesztésével, amely a tantárgy legfontosabb témaköreit interaktív, vizualizációra épülő formában dolgozza fel. A rendszer célja nem csupán az elméleti anyag bemutatása, hanem annak szemléltetése és a gyakorlati megértés támogatása is.

1.2. Célkitűzések

A szakdolgozat célja egy interaktív és vizualizációs oktatási webalkalmazás tervezése és fejlesztése, amely támogatja a hallgatókat a Számításelmélet alapjai I-II tantárgy témaköreinek elsajátításában. A fejlesztés során olyan rendszer létrehozása a cél, amely egyetlen felületen egyesíti az elméleti háttéranyagot, a vizuális szemléltetést és az interaktív gyakorlási lehetőségeket. A hallgatók így nem csupán olvashatnak az egyes modellekről és algoritmusokról, hanem működésüket lépésről lépésre követhetik és saját bemenetek segítségével ki is próbálhatják. A rendszer fontos célja továbbá, hogy támogassa az önálló tanulást, valamint segítséget nyújtson a gyakorlati felkészülésben és a vizsgákra való felkészülés során. A kitűzött célok az alábbiakban foglalhatók össze:

A konkrét célok:

- a Számításelmélet alapjai I–II. főbb témaköreinek feldolgozása;
- interaktív és vizualizációs tanulási modulok kialakítása;
- egyszerűen kezelhető, áttekinthető webes felület létrehozása;
- magyar nyelvű tananyag, a tartalommodellt pedig úgy kialakítani, hogy később angol fordítással is bővíthető legyen;
- bővíthető, moduláris alkalmazásarchitektúra kialakítása.

1.3. Hatókör és korlátok

1.3.1. Hatókör

A szakdolgozat egy webalapú oktatási alkalmazás tervezését és megvalósítását foglalja magában. Az alkalmazás a Számításelmélet alapjai I–II. tárgy meghatározott témaköreit dolgozza fel, és ezekhez elméleti összefoglalót, vizualizációs elemeket és/vagy interaktív funkciókat biztosít. A rendszer támogatja a különböző automatamodellek és algoritmusok működésének szemléltetését, lehetőséget ad a felhasználói bemenetek kiértékelésére, és megjeleníti az eredményeket.

A dolgozat része a rendszer architektúrájának bemutatása, a tervezés dokumentálása, az implementáció ismertetése, valamint a megvalósított funkciók tesztelése és értékelése. Végül egy felhasználói dokumentáció is készül, amely a rendszer használatát segíti.

1.3.2. Korlátok

A dolgozat keretében megvalósított rendszer oktatási célú prototípusnak tekinthető. Bár alkalmas a tanulás támogatására, nem célja egy teljes körű e-learning vagy tanulmányi rendszer kiváltása. A tantárgy minden részterületének teljes körű feldolgozása a rendelkezésre álló idő és erőforrások miatt nem valósítható meg, ezért a fejlesztés elsősorban a legfontosabb és leginkább vizualizálható témakörökre koncentrál. A rendszer egyetlen felhasználó általi használatra készül, így nem tartalmaz közösségi vagy együttműködést támogató funkciókat.

1.3.3. Feltételezések

A rendszer használata modern webböngésző meglétét feltételezi. A felhasználók rendelkeznek alapvető informatikai ismeretekkel, valamint a Számításelmélet alapjai I–II. tárgy követéséhez szükséges előismeretekkel. A tervezés során feltételezem, hogy a webes technológiák és a kliensoldali vizualizációs megoldások elegendő teljesítményt biztosítanak az interaktív modulok futtatásához.

1.4. A dolgozat fő hozzájárulásai

- egy interaktív, webalapú oktatási rendszer tervezése és megvalósítása a Számításelmélet alapjai I–II. tárgy támogatására;
- vizualizációra épülő megközelítés alkalmazása az absztrakt elméleti fogalmak szemléltetésére;
- a tananyag egységes, típusos, blokkalapú tartalommodellbe szervezése;
- moduláris és bővíthető szoftverarchitektúra kialakítása;
- interaktív szimulációs és kiértékelő mechanizmusok fejlesztése;
- a rendszer tervezésének, implementációjának és tesztelésének dokumentálása.

A dolgozat eredménye egy olyan oktatási segédeszköz, amely a hagyományos tananyagokat kiegészítve támogatja az önálló tanulást, a szemléletesebb megértést és a gyakorlati tapasztalatszerzést.

2. fejezet

Felhasználói dokumentáció

TODO

3. fejezet

Fejlesztői dokumentáció

3.1. Követelményelemzés

3.1.1. A rendszer célja és hatóköre

A szakdolgozat hatókörébe egy webalapú, oktatást támogató alkalmazás tervezése és megvalósítása tartozik, amely a Számításelmélet alapjai I–II. tárgy egyes témaköreinek megértését segíti interaktív és vizuális eszközökkel. A 3.1. táblázat a hatókörbe tartozó funkciókat és azok jelenlegi megvalósítási állapotát foglalja össze.

3.1.2. Hatókörön kívül eső elemek

3.1.3. Áttekintés

Mivel az alkalmazás sok témakört dolgoz fel, moduláris felépítésű: az egyes témakörök önálló tanulási egységként fejleszthetők, a rendszer pedig ezekből az egymásra épülő egységekből áll össze. Minden témakörhöz tartozik elméleti összefoglaló, és kapcsolódhat hozzá vizualizáció, illetve interaktív feladat.

A rendszer négy fő modulból áll:

Tananyag modul. A témakörök elméleti anyagának tárolása, elérése és megjelenítése. A tananyag strukturált, típusos adatként tárolódik (lásd 3.2. szakasz), így a rendszer könnyen bővíthető új témakörökkel.

Vizualizációs modul. Az automaták, gráfok és egyéb modellek állapotainak és működésének grafikus megjelenítése.

Funkció	**Leírás**	**Állapot**
Témakörök elérése	A tárgy témakörei egy főoldalon listázódnak, és mindegyik saját oldalon nyitható meg.	kész
Elméleti tananyag	Minden témakörhöz blokkalapú elméleti összefoglaló tartozik (címsor, bekezdés, definíció, tétel, példa, képlet, lista, táblázat).	folyamatban
Automaták szerkesztése	A felhasználó grafikus felületen hozhat létre, szerkeszthet és törölhet DVA és NDVA automatákat.	tervezett
Interaktív vizualizáció	A rendszer vizuálisan jeleníti meg az állapotokat, az átmeneteket és az algoritmusok működését.	tervezett
Algoritmusszimuláció	A felhasználó lépésenként követheti az automaták működését egy bemeneti szó feldolgozása közben.	tervezett
Gyakorló feladatok	A hallgató gyakorló feladatokat oldhat meg, amelyekre az alkalmazás visszajelzést ad.	tervezett

3.1. táblázat. A rendszer hatókörébe tartozó funkciók

Terület	Leírás
Teljes oktatási keretrendszer	Az alkalmazás nem helyettesíti az egyetemi oktatási rendszereket, az előadásokat, a gyakorlatokat vagy az oktatói magyarázatokat.
Felhasználókezelés	Az alkalmazás nem tartalmaz regisztrációs, bejelentkezési vagy profilkezelési funkciókat.
Felhasználói adatok és előrehaladás	A rendszer nem tárol adatot a felhasználók aktivitásáról, a megoldott feladatokról vagy az elért eredményekről, és nem készít egyéni statisztikákat.
Mobilalkalmazás	A rendszer böngészőn keresztül használható, natív mobilalkalmazás nem készül.
Valós idejű együttműködés	A rendszer nem támogatja több felhasználó egyidejű munkáját vagy a közös szerkesztést.

3.2. táblázat. A rendszer hatókörén kívül eső funkciók

Szimulációs modul. Az algoritmusok és modellek lépésenkénti futtatása saját bemenetre, előre- és visszalépési lehetőséggel.

Gyakorló modul. Feladatok megoldása és ellenőrzése, a megoldásra adott visszajelzés.

A modulok egy React-alapú webalkalmazás keretében működnek együtt. Az alkalmazás teljes egészében a kliensoldalon fut, így használatához nincs szükség külön szerveroldali komponensre. A moduláris felépítés a kitűzött célok szempontjából azért előnyös, mert új témakörök és modulok a meglévő komponensek jelentős módosítása nélkül adhatók hozzá.

3.1.4. Külső interfészek

3.1.5. Funkcionális követelmények

FK-1 A rendszer biztosítsa a Számításelmélet alapjai I–II. tárgy témaköreinek elérését és kiválasztását.

FK-2 Minden témakörhöz tartozzon elméleti összefoglaló és magyarázó leírás.

FK-3 Minden témakörhöz kapcsolódjon legalább egy vizualizációs vagy interaktív tanulási modul.

Interfész	Leírás
Böngésző	A felhasználó webböngészőn keresztül éri el a rendszert.
Vizualizációs felület	Az automaták és gráfok grafikus megjelenítését végzi.
Matematikai megjelenítés	A LaTeX szintaxisban tárolt képletek megjelenítését biztosítja (tervezetten KaTeX segítségével).
Helyi tartalomforrás	A tananyag az alkalmazással együtt szállított, lokális adatként érhető el; külső szerver vagy adatbázis nem szükséges.

3.3. táblázat. A rendszer külső interfészei

FK-4 A rendszer jelenítse meg a vizualizációk egyes lépéseit.

FK-5 A felhasználó a vizualizáció állapotai között előre és hátra tudjon lépkedni.

FK-6 A felhasználó saját bemeneteket adhasson meg a vizualizációk számára.

FK-7 A rendszer dolgozza fel és értékelje ki a felhasználó által megadott bemeneteket.

FK-8 A rendszer a kiértékelés eredményét egyértelműen jelenítse meg.

FK-9 Az interaktív modulok tegyék lehetővé különböző számításelméleti modellek és algoritmusok kipróbálását.

FK-10 A felhasználó egy adott algoritmus működését lépésenként követhesse nyomon.

3.1.6. Nem funkcionális követelmények

NFK-1 A rendszer válaszideje normál használat mellett ne haladja meg az 1 másodpercet.

NFK-2 A felhasználói felület legyen modern, áttekinthető és intuitív.

NFK-3 Az alkalmazás különböző képernyőméreteken reszponzívan működjön.

NFK-4 A tananyag magyar nyelven készüljön, a tartalommodell pedig tegye lehetővé az angol nyelvű fordítás későbbi hozzáadását.

NFK-5 A rendszer legyen egyszerűen telepíthető és futtatható különböző környezetekben.

NFK-6 A forráskód legyen jól strukturált, dokumentált és karbantartható.

NFK-7 Az alkalmazás legyen könnyen bővíthető új témakörökkel és tanulási modulokkal.

NFK-8 A vizualizációs és szimulációs komponensek működése legyen tesztekkel ellenőrizhető.

3.2. Adatkövetelmények: a tartalommodell

A tartalommodell kialakítása a az előadásdiák elemzésével történt. A tervezés fő szempontjai a következők voltak:

- a tartalom legyen független a megjelenítési technológiától;
- a tananyag legyen könnyen bővíthető;
- a teljes tananyag egységes formátumban tárolódjon;
- a későbbi interaktív modulok (DVA, NDVA, veremautomata stb.) ráépíthetők legyenek.

A blokkalapú modell előnye, hogy minden témakör ugyanazzal a megjelenítő komponenssel rajzolható ki, új tartalom felvétele egyszerű, és egy későbbi export (például PDF vagy nyomtatható nézet) is könnyen megoldható.

A tananyagot TypeScript-modulokban tárolom. A szerkezet JSON-kompatibilis (csak objektumokat, tömböket, szövegeket és számokat használ), de a TypeScript-típusok miatt a fordító már szerkesztés közben jelzi, ha egy blokk hiányos vagy hibás.

3.2.1. Többnyelvű szöveg

Minden megjelenő szöveg `LocalizedText` típusú. A magyar változat kötelező, az angol opcionális, így a fordítás a meglévő tartalom módosítása nélkül, fokozatosan pótolható.

```
type LocalizedText = {
  hu: string;
  en?: string;
};
```

3.2.2. Tartalomblokkok

Egy témakör elméleti anyaga `ContentBlock` elemek sorozata. A `ContentBlock` ún. diszkriminált unió (*discriminated union*): minden blokknak van egy `type` mezője, amely alapján a fordító és a megjelenítő is el tudja dönteni, milyen további mezői vannak. A jelenleg definiált blokktípusokat a 3.4. táblázat mutatja.

`type`	Mezők	Szerep
`heading`	`level` (1–3), `text`	Fejezet- vagy szakaszcím
`paragraph`	`text`	Folyó szöveg
`definition`	`term`, `content`	Fogalom definíciója
`theorem`	`kind`, `name?`, `statement`, `proof?`	Tétel, lemma, állítás vagy következmény
`example`	`title?`, `content`	Kidolgozott példa
`formula`	`latex`	Kiemelt képlet
`list`	`ordered`, `items`	Számozott vagy felsorolásos lista
`table`	`headers`, `rows`	Táblázat (pl. átmenettáblázat)

3.4. táblázat. A tartalomblokkok típusai (a `?` opcionális mezőt jelöl)

A `theorem` blokk `kind` mezője a `tetel`, `lemma`, `allitas` és `kovetkezmeny` értékek egyikét veheti fel. Egy definíció például így néz ki:

```
{
  type: "definition",
  term: { hu: "Determinisztikus véges automata" },
  content: { hu: "Az $A = \\langle Q, \\Sigma, \\delta, q_0, F \\
     rangle$ rendezett ötös, ahol ..." },
}
```

A tervek szerint a modell további blokktípusokkal bővül: megjegyzés (`note`), lépésekre bontott algoritmus (`algorithm`), valamint egy interaktív automatát beágyazó blokk (`automaton`), amely a tananyagot a szimulációs modullal köti össze.

3.2.3. Matematikai jelölések

A tárgy anyaga sok matematikai jelölést tartalmaz (automaták definíciói, grammatikák, pumpálási lemmák, a CYK algoritmus stb.), ezért a képleteket egységesen LaTeX szintaxissal tárolom:

- szövegen belüli képlet: `$...$` a szövegmezőben;
- kiemelt képlet: külön `formula` blokk;
- a sztringekben a \ karaktert duplán kell írni (\\).

3.2.4. Témakörök metaadatai

A tartalom és a navigáció szétválasztása érdekében minden témakörhöz külön metaadat (`TopicMeta`) tartozik. Ez szolgál a navigációhoz, a sorrendezéshez, a csoportosításhoz, az előadásokhoz kötéshez, és később ezen keresztül kapcsolódik a témakörhöz az interaktív modul.

Mező	Típus	Jelentés
`id`	`string`	Egyedi azonosító, az URL-ben is ez szerepel
`order`	`number`	Megjelenési sorrend
`group`	`TopicGroup`	Témakörcsoport
`title`	`LocalizedText`	Cím
`summary`	`LocalizedText`	Rövid összefoglaló
`lectures`	`number[]`	Kapcsolódó előadások sorszáma
`module`	`string | null`	A kapcsolódó interaktív modul azonosítója

3.5. táblázat. A `TopicMeta` mezői

A témakörök a tárgy logikai felosztását követve öt csoportba (`TopicGroup`) sorolódnak:

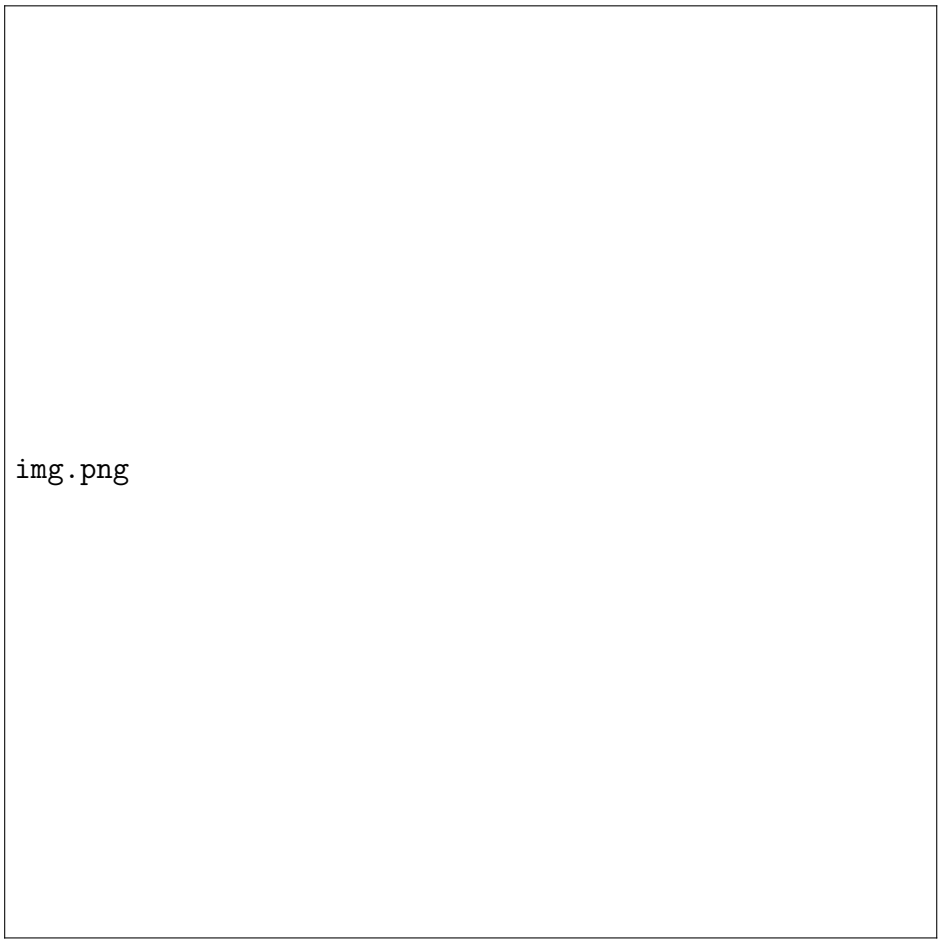
Azonosító	Jelentés
`foundations`	Alapfogalmak
`regular`	Reguláris nyelvek
`context-free`	Környezetfüggetlen nyelvek
`context-sensitive`	Környezetfüggő és 0-típusú nyelvek
`computability`	Kiszámíthatóság (II. rész, későbbi bővítés)

3.6. táblázat. Témakörcsoportok

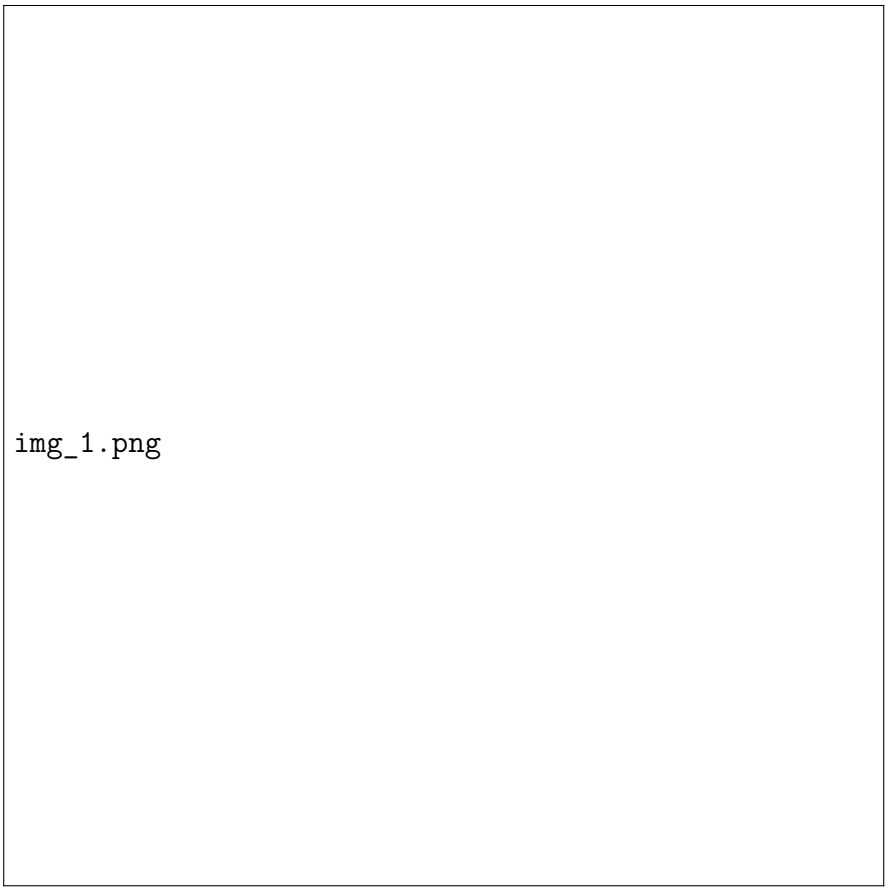
A tárgy I. részéhez jelenleg 14 témakör váza készült el (alapfogalmak, grammatikák, reguláris grammatikák, reguláris kifejezések, DVA, NDVA, minimalizálás, reguláris kifejezés és automata kapcsolata, kis Bar-Hillel-lemma, környezetfüggetlen grammatikák, CYK algoritmus, veremautomata, nagy Bar-Hillel-lemma, környezetfüggő nyelvek). Ezekben egyelőre a metaadatok és a fejezetcímek szerepelnek, a tényleges elméleti szöveg feltöltése a következő fejlesztési fázis feladata.

3.3. Felhasználói felület modell

A felhasználói felület tervezésénél a fő szempont az áttekinthetőség és a könnyű navigáció volt.



3.1. ábra. A főoldal vázlata



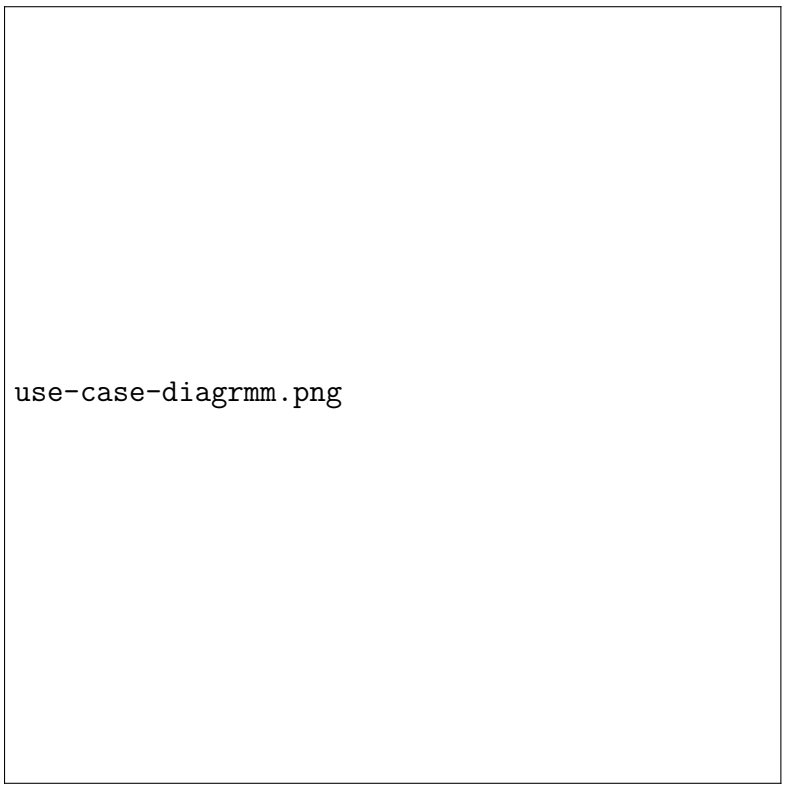
3.2. ábra. Egy témakör oldalának vázlata

3.4. Használati esetek

A rendszer elsődleges szereplője a **Hallgató**, aki a webes felületen keresztül éri el a tárgy tanulását támogató modulokat. A hallgató kiválaszthatja a számára érdekes témakört, megtekintheti az elméleti összefoglalót, használhatja a vizualizációkat, és interaktív feladatokon keresztül gyakorolhatja a tanultakat.

A vizualizációk segítségével a hallgató lépésről lépésre követheti az egyes algoritmusok és modellek működését. Az interaktív feladatoknál saját bemenetet adhat meg, amelyre a rendszer automatikusan kiértékelést és visszajelzést ad.

A **??**. ábra a rendszer fő használati eseteit és azok kapcsolatait mutatja be.



3.3. ábra. A rendszer használati esetei

4. fejezet

Rendszerarchitektúra

4.1. Technológiai választások

Technológia	**Szerep**
TypeScript	A teljes kódbázis és a tananyag nyelve; a típusok a tartalom helyességét is ellenőrzik.
React 19	A felhasználói felület komponensalapú felépítése.
React Router 7	Kliensoldali útvonalkezelés a főoldal és a témakörök oldalai között.
Vite	Fejlesztői szerver és éles build.
pnpm workspaces	Monorepo-kezelés: több, egymásra hivatkozó csomag egy repositoryban.

4.1. táblázat. A felhasznált főbb technológiák

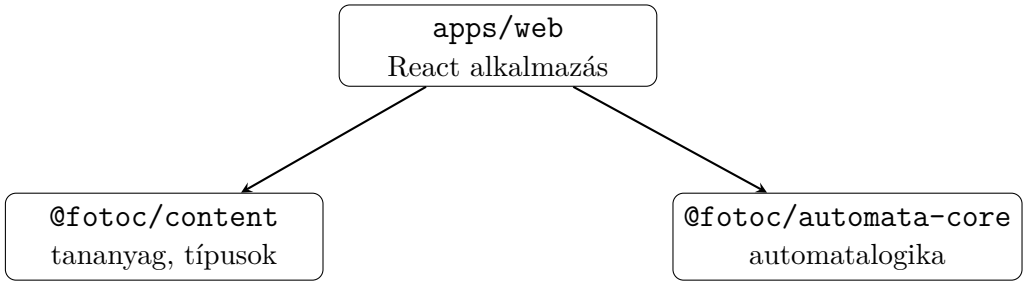
4.2. A kódbázis felépítése

A projekt egy pnpm monorepo, amely egy alkalmazásból és két könyvtárcsomagból áll:

```
Fotoc-helper/
|-- apps/
|   `-- web/                  React webalkalmazás
|       `-- src/
|           |-- main.tsx      belépési pont
|           |-- router/       útvonalak
```

```
|           |-- pages/        oldalak (főoldal, témakör)
|           `-- components/   újrafelhasználható komponensek
|-- packages/
|   |-- content/              @fotoc/content: tananyag és típusai
|   |   `-- src/
|   |       |-- types/        content.ts, topics.ts
|   |       `-- topics/       témakörönként meta.ts + theory.ts,
|   |                         valamint index.ts (registry)
|   `-- automata-core/        @fotoc/automata-core: automatalogika
`-- docs/                     dokumentáció
```

A csomagok közötti függőségeket a **??**. ábra mutatja. A tananyag és az automaták logikája nem függ a React-től, így ezek önállóan, böngésző nélkül is tesztelhetők, és a megjelenítési réteg lecserélhető anélkül, hogy a tartalomhoz hozzá kellene nyúlni.



4.1. ábra. A csomagok közötti függőségek

4.3. A tartalomcsomag (`@fotoc/content`)

A csomag a 3.2. szakaszban leírt típusokat és a témakörök adatait tartalmazza. Minden témakör saját mappát kap két fájllal: a `meta.ts` a metaadatokat, a `theory.ts` az elméleti anyag blokkjait exportálja.

A témakörök összegyűjtését a `topics/index.ts` végzi (*topic registry*). Ez importálja az összes témakört, a `order` mező szerint rendezi őket, és a csomag két belépési pontot ad a webalkalmazásnak:

- `topics`: az összes témakör rendezett listája;
- `getTopic(id)`: egy témakör lekérése azonosító alapján (`undefined`, ha nem létezik).

A registry jelenleg kézzel karbantartott lista, ami a prototípus szakaszban egyszerű és átlátható megoldás. Egy új témakör felvétele tehát három lépésből áll: új mappa `meta.ts` és `theory.ts` fájllal, majd a két import és a bejegyzés felvétele az `index.ts`-be. Később ez a Vite `import.meta.glob` funkciójával automatizálható.

4.4. A webalkalmazás (`apps/web`)

4.4.1. Útvonalak

Az alkalmazás a React Router segítségével két oldalt szolgál ki:

Útvonal	**Komponens**	**Feladat**
`/`	`Homepage`	Az összes témakör listázása, mindegyik hivatkozással a saját oldalára.
`/topics/:topicID`	`Topicpages`	Egy témakör címének, összefoglalójának, csoportjának, a kapcsolódó előadásoknak és az elméleti anyagnak a megjelenítése.

4.2. táblázat. Az alkalmazás útvonalai

A témakör oldala az URL-ből kapott azonosítóval hívja meg a `getTopic` függvényt. Ha ilyen azonosítójú témakör nincs, hibaüzenetet jelenít meg.

4.4.2. Az elméleti anyag megjelenítése

Az elméleti blokkokat a `TheoryRenderer` komponens rajzolja ki. A komponens végigmegy a témakör blokkjain, és mindegyikhez a `type` mező alapján választja ki a megfelelő HTML-elemet. Mivel a `ContentBlock` diszkriminált unió, a TypeScript az egyes ágakban tudja, milyen mezői vannak az adott blokknak (például a `heading` ágban elérhető a `level` mező, a `formula` ágban a `latex`). Egy új blokktípus bevezetésekor így elég a típust és a megjelenítő egy új ágát megírni.

Jelenleg a címsor és a bekezdés blokkok megjelenítése készült el, a többi blokktípus kirajzolása és a képletek KaTeX-alapú megjelenítése a következő fejlesztési lépés.

4.5. Az automatacsomag (`@fotoc/automata-core`)

Ez a csomag fogja tartalmazni az automaták és algoritmusok megjelenítéstől független logikáját (például egy DVA lépésenkénti futtatását egy bemeneti szón). A csomag szerkezete és a webalkalmazással való kapcsolata már kialakításra került, a tényleges implementáció a szimulációs modul fejlesztésével együtt készül el.

Irodalomjegyzék

[1] seanyasno: *Handling icons in React: best practices*. DEV Community. https://dev.to/seanyasno/handling-icons-in-react-best-practices-22c5